

API Documentation

Skill API calls, contracts, examples, necessity, and integration guidance

Purpose and scope

This document is the developer reference for the BeeTrust TypeScript API. It explains what each public Skill call does, why the call is needed in the release workflow, the exact input and output shape, normal and exception behavior, and the recommended way to compose the calls. The public surface is re-exported from `apps/web/src/public-api.ts` and compiled into `apps/web/dist` by `npm run build`.

The package is a local, deterministic prototype. It has no REST server in the repository; consumers import the functions from the compiled TypeScript/JavaScript package or call the supplied command-line demos. The only optional network operation is the live NZ Customs tariff lookup. No call submits a TSW declaration or issues an MPI, laboratory, or Customs certificate.

Supported judged case

Release 1000 jars of UMF Mānuka honey from New Zealand to Australia for AU retail review.

The default case is intentionally constrained so that the API contract, fixture evidence, and acceptance tests remain repeatable. A production integration should replace the synthetic fixture with authorised case data and current broker, MPI, tariff, and laboratory evidence.

Status model

Value	Meaning	Consumer action
PASS	The Skill completed its gate without a blocking issue.	Continue to the next dependency, while retaining evidenceRefs.
REVIEW	The Skill found an unresolved but potentially recoverable issue.	Pause automatic release and request broker, operator, or source verification.
BLOCKED	Required input, evidence, integrity, or fault control failed.	Do not release; resolve the listed reason and run the affected gate again.
RELEASE	The final evidence-monitor decision when all five release gates pass.	Release only to broker review with the evidence bundle; human authority remains explicit.

Quick start

```
import { runTradeCase } from "../dist/public-api.js";

const result = await runTradeCase(undefined, {
  liveSources: false,
  includeRedTeam: true,
  runtimeProvider: "langgraph-stategraph"
});

console.log(result.baseline.decision, result.baseline.score);
console.log(result.tradeCase.monitor?.nextActions);
```

Expected baseline: RELEASE 100/100 for the supported fixture. The returned object also contains the TaskPlan, five-gate baseline, six red-team decisions, runtime summary, SkillMessage/v1 messages, and pilot-labelled KPI proxies.

Public API map and shared contract

Every call below is exported from `apps/web/src/public-api.ts` unless explicitly marked as a type or an internal helper. The API is deliberately function-based so a dashboard, command-line runner, test, or future transport can use the same contract.

API group	Public calls	Why the group exists
Case and workflow	<code>createDemoCase</code> , <code>runTradeCase</code> , <code>parseIntent</code> , <code>createTaskPlan</code> , <code>executeTaskPlan</code> , <code>observeSwarm</code> , <code>renderPlanLevels</code> , <code>renderRuntimeTimeline</code>	Creates one case, plans dependencies, executes work, and makes concurrency and lifecycle trace visible.
Scientific evidence	<code>extractFeatures</code> , <code>compareFingerprint</code>	Normalizes laboratory extracts and turns comparison into a repeatable evidence gate.
Custody integrity	<code>buildHashChain</code> , <code>verifyHashChain</code> , <code>runCustodySkill</code>	Makes payload and batch-identity changes observable through linked hashes.
Market access	<code>checkMpiMarketAccess</code>	Checks destination-specific MPI evidence and freshness before Customs preparation.
Customs preparation	<code>fetchOfficialTariffSource</code> , <code>hsCandidatesFor</code> , <code>runCustomsSkill</code>	Separates classification candidates, transparent cost estimates, source checks, and drafts.
Adversarial testing	<code>ALL_FAULTS</code> , <code>injectFault</code> , <code>evaluateFault</code> , <code>runAdversarySkill</code>	Proves that controlled changes are detected without mutating the baseline case.
Release monitoring	<code>auditTradeCase</code> , <code>renderAuditSummary</code>	Aggregates five gates and applies BLOCKED > REVIEW > RELEASE precedence.
Integration and validation	<code>routeTriggers</code> , <code>runFrameworkRuntime</code> , <code>toAutoGenTeamConfig</code> , <code>toAgentVerseTeamConfig</code> , <code>validateFrameworkMessages</code> , <code>runAcceptance</code> , <code>runAllSelfTests</code> , <code>calculateBusinessKpis</code>	Connects trigger routing, framework-compatible execution, protocol validation, and verification reporting.

Shared input and output conventions

Convention	Contract
Case identity	<code>caseId</code> identifies the trade case; <code>batchId</code> identifies the physical batch; <code>correlationId</code> links a task's lifecycle messages.
Evidence	Successful gates expose <code>evidenceRefs</code> . The final <code>evidenceMatrix</code> retains those references by gate; an empty <code>evidenceRefs</code> list is treated as missing evidence.
Errors	Domain problems are normally represented by REVIEW or BLOCKED results. Orchestration handler failures may emit <code>TASK_RETRY</code> and <code>TASK_FAILED</code> lifecycle messages or reject the Promise after the retry budget.
Determinism	Normal demos use deterministic fixtures and source snapshots. <code>liveSourceLookup=true</code> performs a real HTTP check and therefore requires network availability.
Mutability	Workflow handlers populate the TradeCase outputs. <code>injectFault</code> deep-clones before mutation; direct hash and evidence calls return new result objects.

Shared result shape

```
type StageStatus = "PASS" | "REVIEW" | "BLOCKED";
type ReleaseDecision = "RELEASE" | "REVIEW" | "BLOCKED";

interface GateEvidence {
  status: StageStatus;
  evidenceRefs: string[];
  reason: string;
}
```

Read every result status and `evidenceRefs` before using the output. A non-empty result object is not evidence of a successful gate.

1. orchestration-hub: case creation, intent parsing, planning, and execution

Necessity. The seven Skills need one case identity, dependency-aware order, retries, concurrency, and a structured lifecycle trace before a release recommendation can be made.

Primary call.

```
createDemoCase(intentText?: string): TradeCase
parseIntent(raw: string): TradeIntent
createTaskPlan(caseId: string, intent: TradeIntent): TaskPlan
runTradeCase(intentText?: string, options?: WorkflowOptions): Promise<WorkflowRun>
```

Parameters

Name	Type	Req.	Meaning
intentText	string	No	Trade sentence. If omitted, the supported New Zealand-to-Australia 1000-jar fixture is used.
options.liveSources	boolean	No	When true, Customs attempts the live public tariff lookup. Default is false.
options.includeRedTeam	boolean	No	When false, skip the six fault replays. Default is true.
options.runtimeProvider	RuntimeProvider	No	langgraph-stategraph or local-agentverse-swarm. Default is langgraph-stategraph.
caseId	string	Yes	Stable case identifier used by createTaskPlan.
intent	TradeIntent	Yes	Parsed product, origin, destination, quantity, unit, priority, and requester.

Input example

```
const intent = parseIntent(
  "Release 1000 jars of UMF Mānuka honey from New Zealand to Australia for AU retail review"
);
const plan = createTaskPlan("case-demo-001", intent);
const run = await runTradeCase(undefined, { includeRedTeam: true });
```

Output example

```
{
  plan: { communicationSchema: "SkillMessage/v1", nodes: 6 },
  baseline: { decision: "RELEASE", score: 100 },
  redTeam: [ { fault: "batch-id-tamper", decision: "BLOCKED" } ],
  kpis: { evidenceCoveragePct: 100, faultDetectionPct: 100,
    parallelRootAgents: 3, criticalPathStages: 4, totalMessages: 12,
    status: "TARGET_FOR_PILOT" }
}
```

Detailed behavior

- parseIntent extracts the trade fields and uses the supported defaults when fields are absent.
- createTaskPlan creates six nodes: three roots, Customs after MPI, adversary after operational outputs, and monitor last.
- runTradeCase executes the StateGraph adapter, retains ordered messages, runs red-team replays, and calculates KPI proxies.
- executeTaskPlan runs same-level nodes concurrently, commits in plan order, retries up to maxAttempts, and observeSwarm exposes the runtime shape.

Usage guidance. Use runTradeCase for the complete workflow; use lower-level calls for custom runners and tests. Preserve the returned plan and messages with the case.

Limits and failure handling. The parser is constrained to the demo contract. A missing handler, dependency cycle, or exhausted retry budget is an execution error, not a release result.

2. fingerprint-evidence: laboratory extract parsing and batch comparison

Necessity. This API is necessary because a release cannot treat a laboratory file as a reliable match without normalizing marker names, checking completeness, calculating similarity, and identifying anomalies. It provides a deterministic triage result that can be included in the evidence matrix.

Primary call.

```
extractFeatures(sampleCsv: string): Record<string, number>
compareFingerprint(input: FingerprintInput): FingerprintResult
```

Parameters

Name	Type	Req.	Meaning
sampleCsv	string	Yes	CSV-like marker rows. Comma, semicolon, tab, and colon separators are accepted.
batchId	string	Yes	Batch being evaluated.
reference	Record<string, number>	Yes	Reference marker values keyed by normalized feature name.
referenceBatchId	string	Yes	Identity of the reference profile.

Input example

```
const input = {
  batchId: "MH-NZ-2026-0917",
  sampleCsv: "methylglyoxal,0.62\nmoisture,0.55\npollenDNA,0.28",
  reference: { methylglyoxal: 0.62, moisture: 0.55, pollenDNA: 0.28 },
  referenceBatchId: "REF-MH-AU-2026-08"
};
const result = compareFingerprint(input);
```

Output example

```
{
  status: "PASS", batchId: "MH-NZ-2026-0917",
  referenceBatchId: "REF-MH-AU-2026-08", similarity: 1, confidence: 1,
  anomalies: [], anomaliesDetected: false,
  features: [{ name: "methylglyoxal", value: 0.62, unit: "mg/kg", tolerance: 0.05 }],
  evidenceRefs: ["lab-evidence_..."],
  disclaimer: "Prototype similarity scoring supports triage..."
}
```

Detailed behavior

- extractFeatures skips blank lines, comments, and a feature header, then normalizes aliases such as MG, DHA, HMF, sugar profile, pollen DNA, moisture, conductivity, and delta13C.
- compareFingerprint calculates per-feature tolerance, relative distance, completeness, similarity, and confidence.
- PASS requires no anomalies and at least 90% completeness. REVIEW is used when similarity is at least 0.85 and completeness is at least 0.75 but PASS conditions are not met. Otherwise the result is BLOCKED.
- The returned features include unit and tolerance so a reviewer can understand the comparison rather than receiving only a score.

Usage guidance. Call extractFeatures when a consumer needs the normalized marker map alone. Call compareFingerprint when the result must become a release gate and carry an evidence reference.

Limits and failure handling. Malformed or incomplete data is not silently accepted. The similarity score is prototype triage support; it is not a laboratory certificate, UMF licence, or MPI assurance.

3. custody-ledger: tamper-evident chain construction and verification

Necessity. This API is necessary because a batch travels through multiple actors and locations. A plain event list does not make payload edits or batch-identity changes obvious. The hash chain creates a reproducible integrity check that can be retained with the release case.

Primary call.

```
buildHashChain(events: CustodyEventInput[]): CustodyResult
verifyHashChain(events: CustodyEvent[]): { valid: boolean; tamperedEventIds: string[]; reason?: string }
runCustodySkill(batchId: string, events: CustodyEventInput[]): CustodyResult
```

Parameters

Name	Type	Req.	Meaning
events	CustodyEventInput[]	Yes	Ordered events with eventId, timestamp, actor, location, action, batchId, quantity, and optional metadata.
batchId	string	Yes for runCustodySkill	Expected physical batch identity. Every event must match it.
previousHash	string	Yes for verifyHashChain	Hash link stored on an already chained event.
hash	string	Yes for verifyHashChain	Hash of the canonical event payload and previous link.

Input example

```
const events = [
  { eventId: "farm-harvest", timestamp: "2026-09-01T08:00:00Z",
    actor: "Waikato Listed Apiary", location: "Waikato, NZ",
    action: "HARVEST", batchId: "MH-NZ-2026-0917", quantity: 1000 }
];
const result = runCustodySkill("MH-NZ-2026-0917", events);
```

Output example

```
{
  status: "PASS", batchId: "MH-NZ-2026-0917",
  events: [{ eventId: "farm-harvest", previousHash: "GENESIS", hash: "...64 hex chars..." }],
  headHash: "...64 hex chars...", tamperedEventIds: [],
  evidenceRefs: ["custody-evidence_..."]
}
```

Detailed behavior

- buildHashChain starts at GENESIS and adds previousHash and hash to each event in order.
- verifyHashChain recomputes the payload hash and link for every event and identifies tamperedEventIds.
- runCustodySkill validates that the list is non-empty and every event uses the expected batchId before building the chain.
- A broken link, edited payload, invalid batch identity, or empty list returns BLOCKED through the skill boundary.

Usage guidance. Use runCustodySkill in a release workflow. Use buildHashChain when constructing a new ledger and verifyHashChain when independently checking a stored ledger or a received event list.

Limits and failure handling. The chain is an evidence-integrity layer, not a public blockchain, ownership registry, or legal title record. Store the full events and headHash together; a headHash without the events is not enough for independent verification.

4. mpi-market-access: destination-specific evidence and rule checks

Necessity. This API is necessary because market-access requirements vary by destination and evidence freshness. It provides check-level results for beekeeper status, harvest declaration, RMP, OMAR, export certificate, trade certification, freshness, and rule-version alignment before Customs documents are prepared.

Primary call.

```
checkMpiMarketAccess(destination: string, evidence: MpiEvidence): MpiResult
```

Parameters

Name	Type	Req.	Meaning
destination	string	Yes	Destination name or supported alias such as Australia or AU.
listedBeekeeper	boolean	Yes	Whether listed beekeeper evidence is available.
harvestDeclaration	boolean	Yes	Whether harvest declaration evidence is available.
rmp	boolean	Yes	Whether the risk management programme evidence is available.
omar	OMAR state	Yes	AVAILABLE, NOT_REQUIRED, MISSING, or EXPIRED.
exportCertificate	boolean	Yes	Whether export certificate evidence is available.
tradeCertification	boolean	Yes	Whether trade certification evidence is available.
declarationDate	string	Yes	ISO-like declaration date retained with the case.
evidenceFreshnessDays	number	Yes	Prototype freshness age; valid range is 0 through 365.
ruleVersion	string	No	Case rule version. A mismatch with the current snapshot is BLOCKED.

Input example

```
const result = checkMpiMarketAccess("Australia", {
  listedBeekeeper: true, harvestDeclaration: true, rmp: true,
  omar: "AVAILABLE", exportCertificate: true, tradeCertification: true,
  declarationDate: "2026-09-05", evidenceFreshnessDays: 12,
  ruleVersion: "MPI-AU-2026.09"
});
```

Output example

```
{
  status: "PASS", destination: "Australia", ruleVersion: "MPI-AU-2026.09",
  checks: [{ id: "omar", status: "PASS", passed: true, evidenceRef: "mpi-evidence_..." }],
  missingEvidence: [], evidenceRefs: ["mpi-evidence_..."],
  sourceUrls: ["https://www.mpi.govt.nz/..."],
  disclaimer: "Rule snapshot is a decision-support prototype..."
}
```

Detailed behavior

- The destination is resolved to a versioned rule snapshot. The supported demo uses MPI-AU-2026.09.
- Each required check returns passed, status, reason, and evidenceRef so the operator can identify the exact missing item.
- Unknown destinations, missing required evidence, expired OMAR, stale or negative freshness, and rule-version mismatch fail closed to BLOCKED.
- A non-required check can produce REVIEW without being silently converted to PASS.

Usage guidance. Call this API before runCustomsSkill in a dependency-aware workflow. Persist ruleVersion, sourceUrls, checks, and evidenceRefs with the case so a later reviewer sees which snapshot was used.

Limits and failure handling. The snapshot is a prototype decision-support layer. It does not issue an MPI assurance, certificate, or legal market-access determination. Current official requirements must be revalidated before a real export.

5. customs-clearance: tariff lookup, classification candidates, cost estimate, and drafts

Necessity. This API is necessary because Customs preparation combines product lines, classification, freight, insurance, rates, and documents. It separates candidate and verified values, records source metadata, and generates consistent broker-review drafts without implying a declaration was submitted.

Primary call.

```
fetchOfficialTariffSource(live?: boolean): Promise<TariffSnapshot>
hsCandidatesFor(product: string): Array<{ code: string; description: string; confidence: number }>
runCustomsSkill(input: CustomsInput): Promise<CustomsResult>
```

Parameters

Name	Type	Req.	Meaning
caseId / seller / buyer	string	Yes	Case identity and parties used in document numbers and the commercial invoice.
origin / destination / currency	string / "NZD"	Yes	Trade route and current currency contract.
lines	CustomsLine[]	Yes	SKU, description, quantity, unit, unitValueNzd, and netWeightKg.
freightNzd / insuranceNzd	number	Yes	Non-negative cost inputs.
dutyRate / levyRate / exportGstRate	number	Yes	Case-supplied non-negative rates for transparent calculation.
classificationEvidence	boolean	Yes	Whether the candidate is supported by broker or tariff evidence.
liveSourceLookup	boolean	No	When true, fetch and content-check the public tariff page; default false.

Input example

```
const result = await runCustomsSkill({
  caseId: "case-demo-001", seller: "Comvita Limited", buyer: "Australian importer",
  origin: "New Zealand", destination: "Australia", currency: "NZD",
  lines: [{ sku: "MH-250-UMF15", description: "UMF Mānuka honey 250g", quantity: 1000, unit: "jars", unitValueNzd: 12, netWeightKg: 250 }],
  freightNzd: 800, insuranceNzd: 100, dutyRate: 0, levyRate: 0.01, exportGstRate: 0,
  classificationEvidence: true, liveSourceLookup: false
});
```

Output example

```
{ status: "PASS", hsCandidates: [{ code: "0409.00", confidence: 0.9 }],
  totals: { goodsValueNzd: 12000, freightNzd: 800, insuranceNzd: 100, dutyNzd: 0, levyNzd: 120, exportGstNzd: 0,
    estimatedLandedValueNzd: 13020 },
  documents: ["COMMERCIAL_INVOICE", "PACKING_LIST", "TSW_DRAFT"],
  validationIssues: [], evidenceRefs: ["customs-evidence..."], assumptions: ["HS code is a candidate..."]
}
```

Detailed behavior

- `fetchOfficialTariffSource(false)` returns a deterministic snapshot; `true` uses a five-second timeout, content marker, HTTP metadata, and SHA-256 contentHash.
- `hsCandidatesFor` returns natural-honey candidates; other products return `VERIFY_REQUIRED`. `runCustomsSkill` calculates totals and creates three draft document objects.
- Missing parties, invalid lines, invalid rates, or required fields produce `BLOCKED`. Weak classification evidence or failed live verification produces `REVIEW`.
- TSW output is always `DRAFT_ONLY` with `readyForBrokerReview=true`; no declaration is submitted.

Usage guidance. Use deterministic mode for tests; `liveSourceLookup=true` may return `REVIEW`. Preserve assumptions with totals.

6. trade-risk-adversary: controlled fault injection and negative-path evaluation

Necessity. This API is necessary because a clean happy path does not prove that a release desk can detect changes after earlier checks passed. The adversarial API tests the six supported incident modes against a clone and preserves the original baseline for comparison.

Primary call.

```
ALL_FAULTS: FaultType[]
injectFault(tradeCase: TradeCase, fault: FaultType): TradeCase
evaluateFault(tradeCase: TradeCase, fault: FaultType): AdversaryFinding
runAdversarySkill(tradeCase: TradeCase, activeFaults?: FaultType[]): AdversaryResult
```

Parameters

Name	Type	Req.	Meaning
tradeCase	TradeCase	Yes	Complete case to clone or evaluate.
fault	FaultType	Yes	One of fingerprint-mismatch, batch-id-tamper, missing-document, quantity-mismatch, destination-change, rule-version-conflict.
activeFaults	FaultType[]	No	Selected faults. Empty or omitted means baseline catalogue mode; non-empty means active incident mode.

Input example

```
const mutated = injectFault(baseCase, "quantity-mismatch");
const finding = evaluateFault(baseCase, "quantity-mismatch");
const incident = runAdversarySkill(baseCase, ["quantity-mismatch"]);
```

Output example

```
{
  finding: { fault: "quantity-mismatch", severity: "CRITICAL",
    observed: "Invoice quantity diverges from the case quantity.",
    expectedGate: "BLOCKED" },
  incident: { status: "BLOCKED", activeFaults: ["quantity-mismatch"],
    findings: [/* finding above */, evidenceRefs: ["red-team-evidence_..."] }
}
```

Detailed behavior

- ALL_FAULTS is the explicit catalogue used by the incident demo and acceptance test.
- injectFault performs a deep clone before changing one field, so the source TradeCase remains unchanged.
- evaluateFault returns a human-readable title, severity, observed behavior, and expectedGate=BLOCKED.
- runAdversarySkill with activeFaults returns BLOCKED and findings. With no active faults it returns a PASS baseline result and a catalogue evidence reference.

Usage guidance. Run the baseline workflow first, then call runAdversarySkill with one or more selected faults. Compare the mutated decision with the original baseline and store both outcomes. Use `npm run demo:incident` for the supplied dashboard flow.

Limits and failure handling. Only the six registered FaultType values are supported. This is controlled resilience testing, not unconstrained chaos engineering. An unknown fault should be rejected by the caller rather than invented dynamically.

7. evidence-monitor: five-gate audit and final release decision

Necessity. This API is necessary because individual PASS results do not guarantee a safe release. The monitor combines scientific, custody, MPI, Customs, and adversarial evidence, applies precedence rules, and returns the reasons and next actions a human reviewer needs.

Primary call.

```
auditTradeCase(tradeCase: TradeCase): MonitorResult  
renderAuditSummary(result: MonitorResult): string
```

Parameters

Name	Type	Req.	Meaning
tradeCase.fingerprint	FingerprintResult	Expected	Scientific gate and evidenceRefs.
tradeCase.custody	CustodyResult	Expected	Chain-of-custody gate and evidenceRefs.
tradeCase.mpi	MpiResult	Expected	Destination market-access gate and evidenceRefs.
tradeCase.customs	CustomsResult	Expected	Classification and document gate and evidenceRefs.
tradeCase.adversary	AdversaryResult	Expected	Baseline stress gate; active faults make it BLOCKED.

Input example

```
const result = auditTradeCase(tradeCase);  
console.log(renderAuditSummary(result));
```

Output example

```
{  
  decision: "RELEASE", score: 100,  
  gates: [  
    { gate: "scientific-fingerprint", status: "PASS", evidenceRefs: ["..."] },  
    { gate: "chain-of-custody", status: "PASS", evidenceRefs: ["..."] },  
    { gate: "mpi-market-access", status: "PASS", evidenceRefs: ["..."] },  
    { gate: "customs-clearance", status: "PASS", evidenceRefs: ["..."] },  
    { gate: "adversarial-stress", status: "PASS", evidenceRefs: ["..."] }  
  ],  
  missingEvidence: [],  
  nextActions: ["Release to broker review with the evidence bundle."],  
  evidenceMatrix: { "scientific-fingerprint": ["..."] }  
}
```

Detailed behavior

- The five gates are always represented in the evidence matrix, even when an upstream output is missing.
- Decision precedence is BLOCKED over REVIEW over RELEASE. A missing output or empty evidenceRefs prevents an unqualified release.
- The score starts at 100 and subtracts 25 for each BLOCKED gate, 10 for each REVIEW gate, and 5 for each missing evidence set, bounded at zero.
- renderAuditSummary provides a concise text view containing decision, score, gate statuses, evidence counts, and next actions.

Usage guidance. Call auditTradeCase only after the required upstream outputs are present, or call it earlier to obtain a fail-closed diagnostic. Treat RELEASE as a recommendation to broker review, not an autonomous legal or regulatory approval.

Limits and failure handling. The monitor cannot validate the truth of an external document; it validates the structured outputs and references supplied to it. Human review remains required for certificates, laboratory judgments, classification, rates, and final submission.

Protocol, framework, and trigger APIs

The following APIs are needed when the Skills are embedded in a dashboard, agent framework, or alternate runtime. They preserve one message contract instead of creating a separate adapter contract for every integration.

SkillMessage/v1

```
interface SkillMessage<T = unknown> {
  sender: SkillName; receiver: SkillName; protocol: "SkillMessage/v1";
  taskState: "PENDING" | "RUNNING" | "SUCCESS" | "RETRY" | "FAILED" | "ERROR";
  id: string; caseId: string; correlationId: string; skill: SkillName;
  type: "TASK_CREATED" | "TASK_STARTED" | "TASK_COMPLETED" |
  "TASK_RETRY" | "TASK_FAILED" | "EVIDENCE_APPENDED";
  timestamp: string; attempt: number; status: StageStatus | "PENDING" | "RUNNING" | "ERROR";
  payload: T; evidenceRefs: string[];
}
```

Each lifecycle message is necessary for reconstructing who ran what, for which case, on which attempt, with which evidence. Consumers should validate protocol, sender, receiver, IDs, taskState, and evidenceRefs before accepting a trace.

Framework calls

Call	Signature	Use
runFrameworkRuntime	runFrameworkRuntime(plan, tradeCase, handlers, provider?): Promise<FrameworkRuntimeResult>	Executes the current LangGraph StateGraph adapter or local AgentVerse-compatible mode and returns ordered messages plus runtime summary.
toAutoGenTeamConfig	toAutoGenTeamConfig(plan): AutoGenTeamConfig	Maps the same DAG to a GraphFlow-compatible team descriptor; it does not start a remote AutoGen runtime.
toAgentVerseTeamConfig	toAgentVerseTeamConfig(plan): AgentVerseTeamConfig	Maps the plan to planner, workers, observer, and shared schema; it is a local compatibility descriptor.
validateFrameworkMessages	validateFrameworkMessages(messages): string[]	Returns validation errors. An empty array means the message envelope passes the local protocol checks.
routeTriggers	routeTriggers(input: string): SkillName[]	Uppercases input and routes all registered trigger words to matching Skills; unknown input returns an empty array.

Framework example

```
const plan = createTaskPlan(caseId, parseIntent(intentText));
const descriptors = toAutoGenTeamConfig(plan);
const errors = validateFrameworkMessages(run.messages);
if (errors.length > 0) throw new Error(errors.join("; "));
```

Validation and measurement calls

Call	Output and necessity
runAllSelfTests	Promise<TestSummary[]>; verifies the seven Skill suites and their local behavior before packaging or release demos.
runAcceptance	Promise<AcceptanceReport>; checks structure, contracts, public source correspondence, runnable examples, live integration, trigger fixtures, E2E, framework bridge, and fail-closed branches.
calculateBusinessKpis	BusinessKpis; reports evidence coverage, fault detection, parallel roots, critical path, messages, and transparent manual/orchestrated minute proxies. Status is TARGET_FOR_PILOT.
renderDashboardHtml	string; creates the self-contained dashboard view of the DAG, gates, runtime, evidence, and red-team results.

These calls are verification and presentation utilities. They do not replace the seven business gates and should not be used to convert a failed gate into a pass.

Recommended usage patterns

Pattern A: complete release workflow

```
import { runTradeCase } from "../dist/public-api.js";

const run = await runTradeCase(
  "Release 1000 jars of UMF Mānuka honey from New Zealand to Australia for AU retail review",
  { liveSources: false, includeRedTeam: true, runtimeProvider: "langgraph-stategraph" }
);

if (run.baseline.decision === "RELEASE") {
  console.log("Send the evidence bundle to broker review");
} else {
  console.log(run.baseline.nextActions);
}
```

Use this pattern for the judged demo and for the highest-level application integration. The function creates the case, executes the dependency-aware workflow, runs the supported red-team catalogue, and calculates the KPI proxy in one call.

Pattern B: direct Skill composition

```
const intent = parseIntent(intentText);
const caseData = createDemoCase(intent.raw);
caseData.fingerprint = compareFingerprint({
  batchId: caseData.batchId, sampleCsv: caseData.sampleCsv,
  reference: caseData.referenceFingerprint,
  referenceBatchId: caseData.referenceBatchId
});
caseData.custody = runCustodySkill(caseData.batchId, caseData.custodyEvents);
caseData.mpi = checkMpiMarketAccess(caseData.destination, caseData.mpiEvidence);
caseData.customs = await runCustomsSkill(caseData.customsInput);
caseData.adversary = runAdversarySkill(caseData);
caseData.monitor = auditTradeCase(caseData);
```

Use direct composition for isolated tests, custom front ends, or controlled replays. Preserve the same output property names and call `auditTradeCase` only after the upstream properties have been populated.

Pattern C: live source lookup

```
const snapshot = await fetchOfficialTariffSource(true);
if (!snapshot.live) {
  console.warn(snapshot.note);
  // Keep the Customs result at REVIEW and request manual verification.
}
```

Live mode is optional and network-dependent. The deterministic default is preferable for repeatable judging. A live failure is meaningful evidence that the source could not be verified, not a reason to assume the tariff is correct.

Pattern D: controlled incident response

```
const baseline = await runTradeCase(undefined, { includeRedTeam: false });
const fault = "destination-change" as const;
const mutated = injectFault(baseline.tradeCase, fault);
const finding = evaluateFault(baseline.tradeCase, fault);
const incident = runAdversarySkill(baseline.tradeCase, [fault]);
const incidentDecision = auditTradeCase({ ...mutated, adversary: incident });
```

Always retain both baseline and incident records. The supported contract expects the baseline to remain `RELEASE` and each active fault to produce a `BLOCKED` finding after downstream evaluation.

Pattern E: command-line verification

```
cd apps/web
npm install
npm run build
npm test
npm run acceptance
npm run demo:live
npm run demo:incident
```

The compiled dashboard is written to `apps/web/dist/beetrust-dashboard.html`. The `dist` directory is generated output; source changes belong under `apps/web/src` and `apps/web/skills`. Keep generated output out of source control when the repository's ignore policy requires it.

Integration checklist and safety boundaries

Before calling	Check
Any Skill	Validate required fields, identify the caseId and batchId, and decide whether deterministic or live source mode is appropriate.
After calling	Read status, validation issues or missingEvidence, evidenceRefs, and disclaimer/assumptions. Do not infer PASS from a populated object alone.
Before release	Require the five monitor gates, a non-empty evidence reference for each passing gate, and human approval for the broker hand-off.
For Customs	Treat HS candidates, rates, landed value, and document objects as draft decision support. TSW_DRAFT must remain DRAFT_ONLY.
For MPI	Refresh current destination rules and official assurance evidence; the bundled versioned snapshot is not a production certificate.
For laboratory data	Use accredited records and qualified review; fingerprint similarity is only deterministic prototype triage.
For custody	Retain ordered events, previousHash, hash, and headHash together so verification can be repeated.
For incidents	Use only ALL_FAULTS values, clone before mutation, and compare the incident result with the untouched baseline.

Worked final decision example

```
const summary = renderAuditSummary(run.baseline);
// Decision: RELEASE
// Score: 100/100
// PASS scientific-fingerprint (1 evidence refs)
// PASS chain-of-custody (1 evidence refs)
// PASS mpi-market-access (7 evidence refs)
// PASS customs-clearance (2 evidence refs)
// PASS adversarial-stress (1 evidence refs)
// Next actions:
// - Release to broker review with the evidence bundle.
// - Retain source snapshots, hashes and generated drafts with the case.
```

Implementation references

The API contract is implemented in apps/web/src/public-api.ts, apps/web/src/types.ts, apps/web/src/workflow.ts, apps/web/src/framework-runtime.ts, apps/web/src/framework-bridge.ts, apps/web/src/kpis.ts, apps/web/src/trigger-registry.ts, and the seven apps/web/src/skills/*/index.ts modules. Canonical Skill contracts, resources, and runnable examples are under apps/web/skills.

Verification evidence

The repository verification workflow is npm run build, npm test, npm run acceptance, npm run demo:live, and npm run demo:incident. The API documentation describes the contract that those commands exercise; it does not substitute static inspection for execution.

Final boundary

BeeTrust returns explainable prototype decisions for broker review. It does not issue a legal release, guarantee market access, certify a laboratory result, determine a final HS classification, submit to TSW, or claim historical enterprise savings. The KPI output is labelled TARGET_FOR_PILOT until operator telemetry is collected.